



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Proyecto Administrador de BD en consola o
termina**

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Jahuira Pilco, Dayan Elvis

(2022075749)

Mamani Cori, Cristhian Carlos

(2023077282)

**Tacna – Perú
2026**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

Sistema Administrador de BD en consola o termina Documento de Arquitectura de Software

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

INDICE GENERAL

Contenido

1. INTRODUCCIÓN	5
1.1. Propósito (Diagrama 4+1)	5
1.2. Alcance	5
1.3. Definición, siglas y abreviaturas	5
1.4. Organización del documento	6
2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS	6
2.1. Priorización de requerimientos	6
2.1.1. Requerimientos Funcionales	6
2.1.2. No Funcionales – Atributos de Calidad	6
2.2. Restricciones	7
3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA	8
3.1. Vista de Caso de uso	8
3.1.1. Diagramas de Casos de uso	8
3.2. Vista Lógica	8
3.2.1. Diagrama de Subsistemas (paquetes)	8
3.2.2. Diagrama de Secuencia (vista de diseño)	9
3.2.3. Diagrama de Colaboración (vista de diseño)	9
3.2.4. Diagrama de Objetos	9
3.2.5. Diagrama de Clases	10
3.2.6. Diagrama de Base de datos (relacional o no relacional)	10
3.3. Vista de Implementación (vista de desarrollo)	10
3.3.1. Diagrama de arquitectura software (paquetes)	10
3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)	11
3.4. Vista de procesos	11
3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)	11
3.5. Vista de Despliegue (vista física)	12
3.5.1. Diagrama de despliegue	12
4. ATRIBUTOS DE CALIDAD DEL SOFTWARE.	12

1. INTRODUCCIÓN

1.1. Propósito (Diagrama 4+1)

El presente documento tiene como propósito definir la arquitectura de software del sistema "Administrador de BD en consola o terminal" utilizando el modelo de vistas 4+1 (Lógica, Implementación, Procesos, Despliegue y Casos de Uso). Presenta una visión global del diseño, justificando cómo las decisiones arquitectónicas satisfacen los requerimientos funcionales de administración de bases de datos y las prioridades de modularidad, extensibilidad y facilidad de uso en entornos de consola.

1.2. Alcance

Este documento se centra en el desarrollo de la arquitectura del sistema CLI en Python, su estructura modular de conectores (relacionales y NoSQL) y el formateo de resultados. Incluye la vista lógica (conectores y REPL), la vista de implementación (paquetes y componentes) y la vista de procesos (flujo de ejecución de comandos). Adicionalmente, cubre la arquitectura de las tres integraciones de distribución del sistema: la extensión de Visual Studio Code, el servidor MCP (Model Context Protocol) y el bot de Telegram, todos reutilizando la misma capa de conectores del CLI. Se omiten procesos de interfaz gráfica ya que el sistema opera completamente en consola.

1.3. Definición, siglas y abreviaturas

CLI: Interfaz de Línea de Comandos (Command Line Interface).

REPL: Bucle de Lectura-Evaluación-Impresión (Read-Eval-Print Loop).

CRUD: Operaciones de Crear, Leer, Actualizar y Eliminar (Create, Read, Update, Delete).

SGBD: Sistema Gestor de Bases de Datos.

SQL: Lenguaje de Consulta Estructurado (Structured Query Language).

NoSQL: Bases de datos no relacionales (Not Only SQL).

CSV: Valores Separados por Comas (Comma-Separated Values).

DDL: Lenguaje de Definición de Datos (Data Definition Language).

DML: Lenguaje de Manipulación de Datos (Data Manipulation Language).

1.4. Organización del documento

El documento está organizado en cuatro secciones principales: Objetivos y restricciones (define qué se debe cumplir), Representación de la arquitectura (donde se exponen los diagramas 4+1), y finalmente los atributos de calidad del software.

2. OBJETIVOS Y RESTRICCIONES ARQUITECTONICAS

2.1. Priorización de requerimientos

2.1.1. Requerimientos Funcionales

ID	Descripción	Prioridad
RF-01	Permitir la conexión multimotor a bases de datos relacionales y no SQL	Muy Alta
RF-02	Crear tablas en la base de datos	Alta
RF-03	Insertar registros en las tablas	Muy Alta
RF-04	Consultar datos almacenados	Muy Alta
RF-05	Actualizar registros existentes	Alta
RF-06	Eliminar registros	Alta
RF-07	Listar tablas existentes	Media
RF-08	Mostrar resultados en consola	Muy Alta

RF-09	Salir del sistema	Baja
-------	-------------------	------

2.1.2. No Funcionales – Atributos de Calidad

ID	Descripción	Prioridad
RNF-01	El sistema debe ejecutarse en entorno de consola (CLI)	Muy Alta
RNF-02	El sistema debe ser compatible con sistemas operativos comunes	Alta
RNF-03	El tiempo de respuesta debe ser menor a 2 segundos	Alta
RNF-04	El sistema debe manejar errores de entrada del usuario	Muy Alta
RNF-05	El código debe estar estructurado y modular	Alta
RNF-06	El sistema debe ser fácil de usar mediante comandos claros	Media
RNF-07	El sistema debe ser construida como un archivo ejecutable independiente, para facilitar la distribución en Windows	Alta
RNF-08	El sistema debe validar la sintaxis básica de los comandos	Muy Alta

2.2. Restricciones

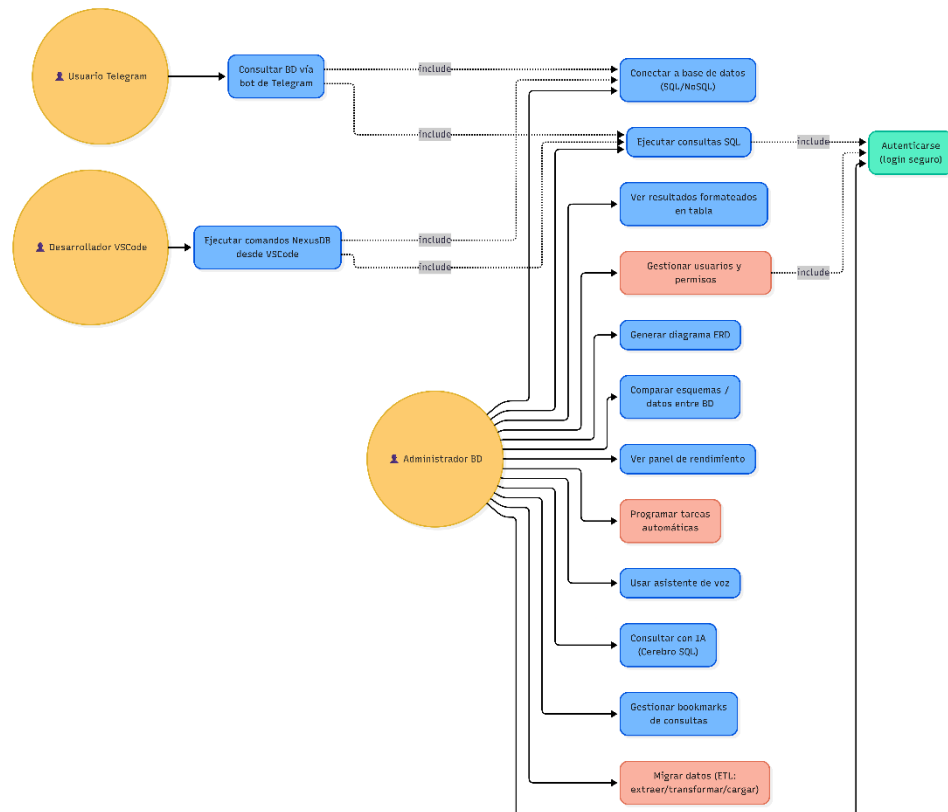
El desarrollo debe utilizar estrictamente Python 3.8 o superior. Las librerías externas permitidas son únicamente aquellas necesarias para la conexión a cada motor de base de datos (relacional y NoSQL), la librería Rich para el formateo de tablas en consola, y las librerías oficiales de las integraciones externas (mcp, python-telegram-bot). La interfaz principal debe ser exclusivamente mediante línea de comandos, sin componentes gráficos. El sistema no debe incluir un motor de base de datos propio, actuando únicamente como intermediario entre el usuario y el SGBD. Las

herramientas expuestas al servidor MCP y al bot de Telegram deben restringirse a operaciones de solo lectura.

3. REPRESENTACIÓN DE LA ARQUITECTURA DEL SISTEMA

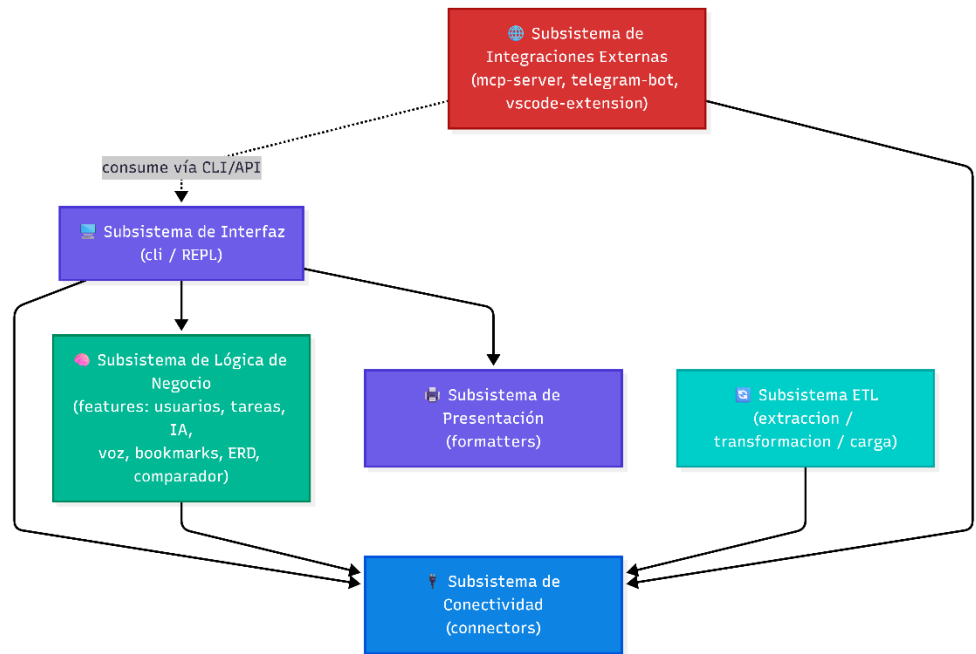
3.1. Vista de Caso de uso

3.1.1. Diagramas de Casos de uso

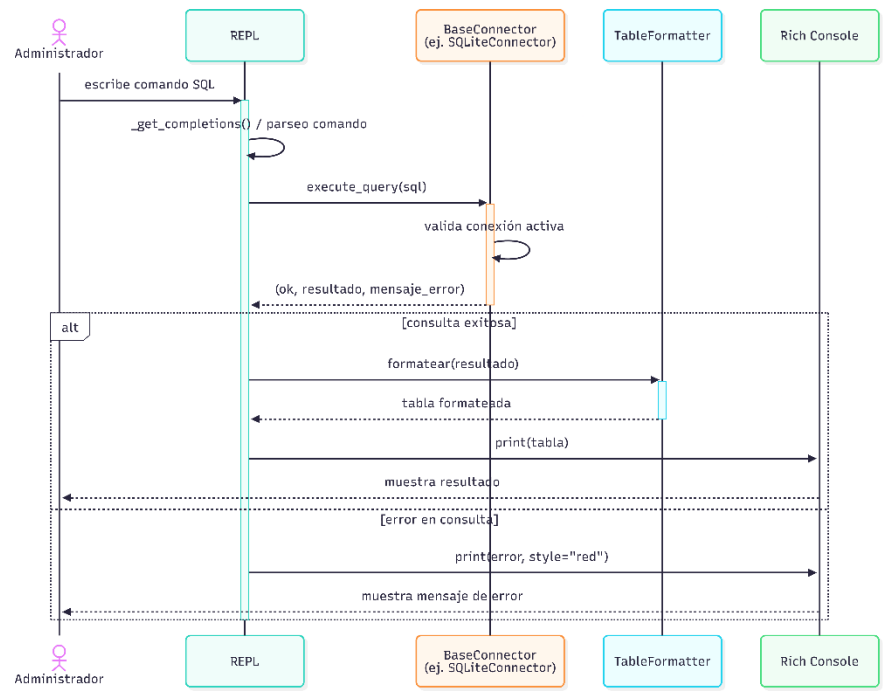


3.2. Vista Lógica

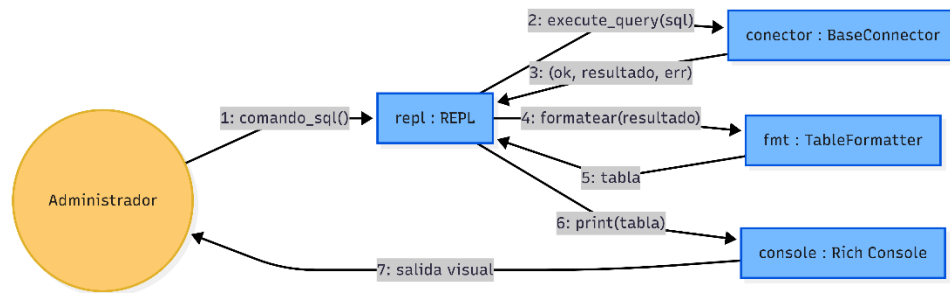
3.2.1. Diagrama de Subsistemas (paquetes)



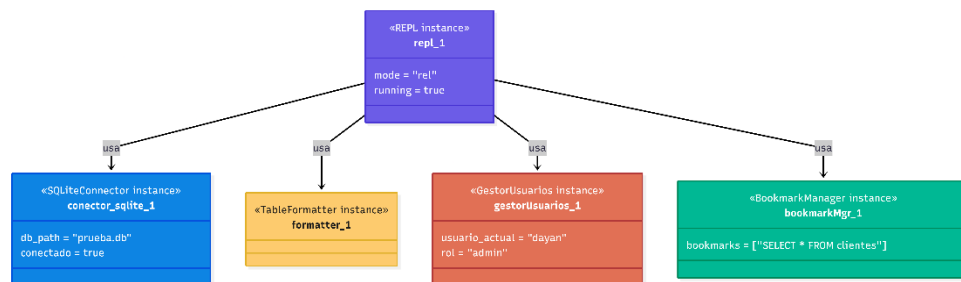
3.2.2. Diagrama de Secuencia (vista de diseño)



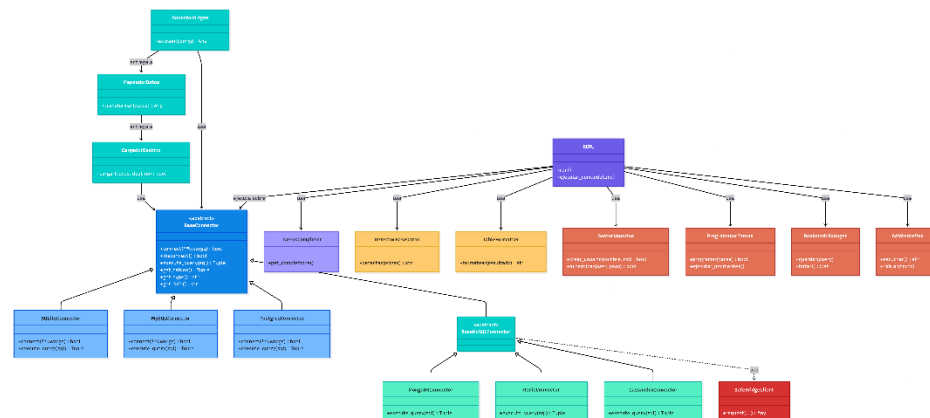
3.2.3. Diagrama de Colaboración (vista de diseño)



3.2.4. Diagrama de Objetos



3.2.5. Diagrama de Clases



3.2.6. Diagrama de Base de datos (relacional o no relacional)

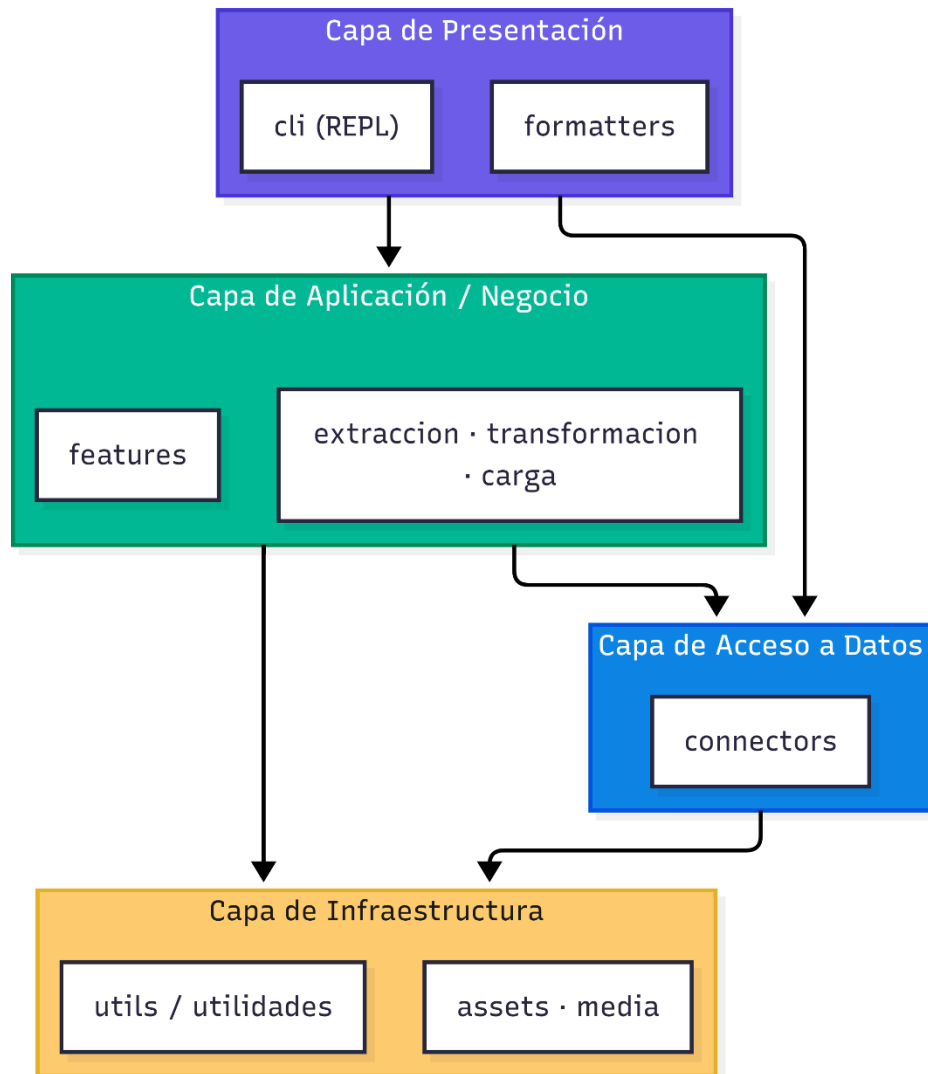
El sistema Administrador de BD en consola o terminal no cuenta con una base de datos propia, ya que su función es actuar como interfaz entre el usuario y los motores de base de datos externos. La arquitectura del sistema está diseñada para conectarse a bases de datos ya existentes sin requerir un esquema de almacenamiento interno.

El único dato que el sistema mantiene en memoria durante la sesión es el resultado de la última consulta SELECT ejecutada, el cual se almacena en la variable `last_results` para permitir su exportación a CSV. Esta

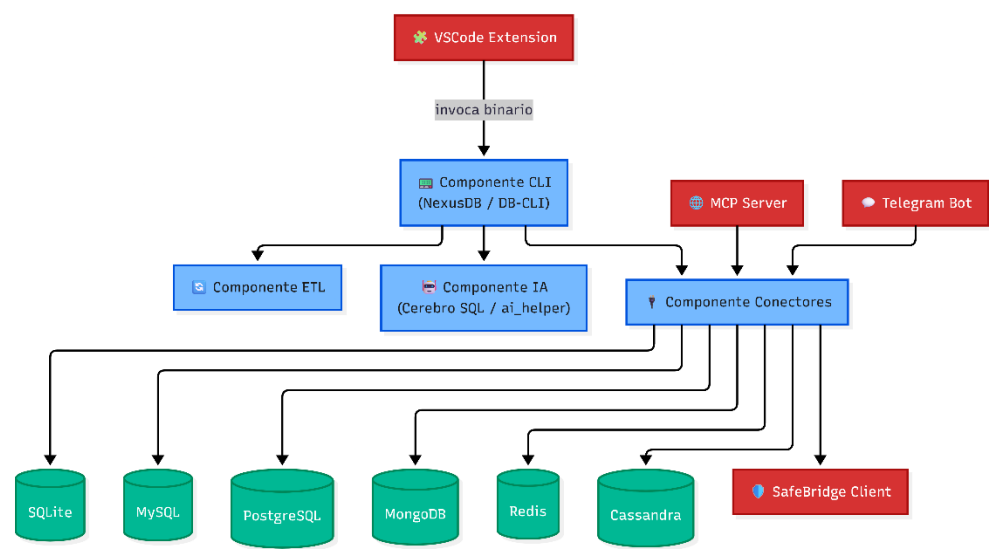
información es volátil y se descarta al cerrar el programa o al ejecutar una nueva consulta.

3.3. Vista de Implementación (vista de desarrollo)

3.3.1. Diagrama de arquitectura software (paquetes)

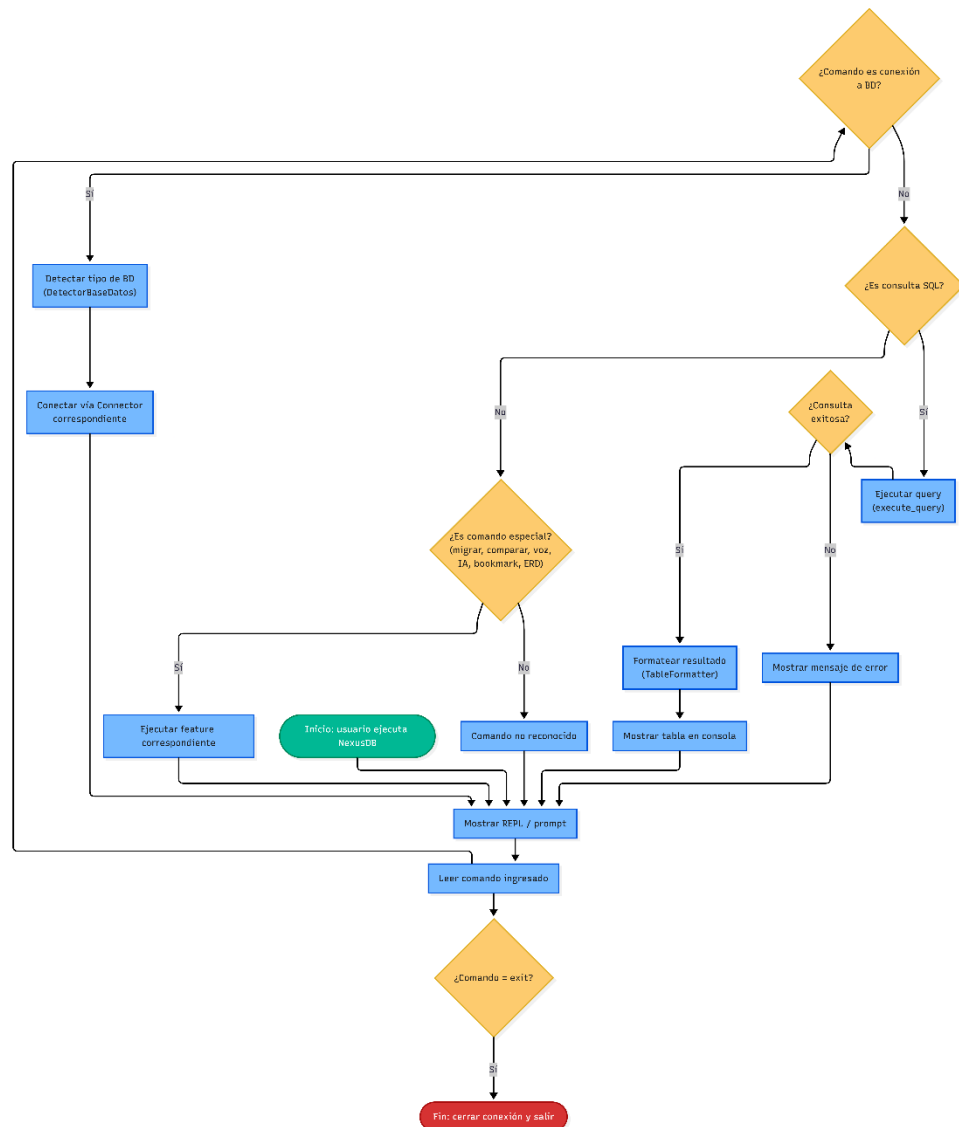


3.3.2. Diagrama de arquitectura del sistema (Diagrama de componentes)



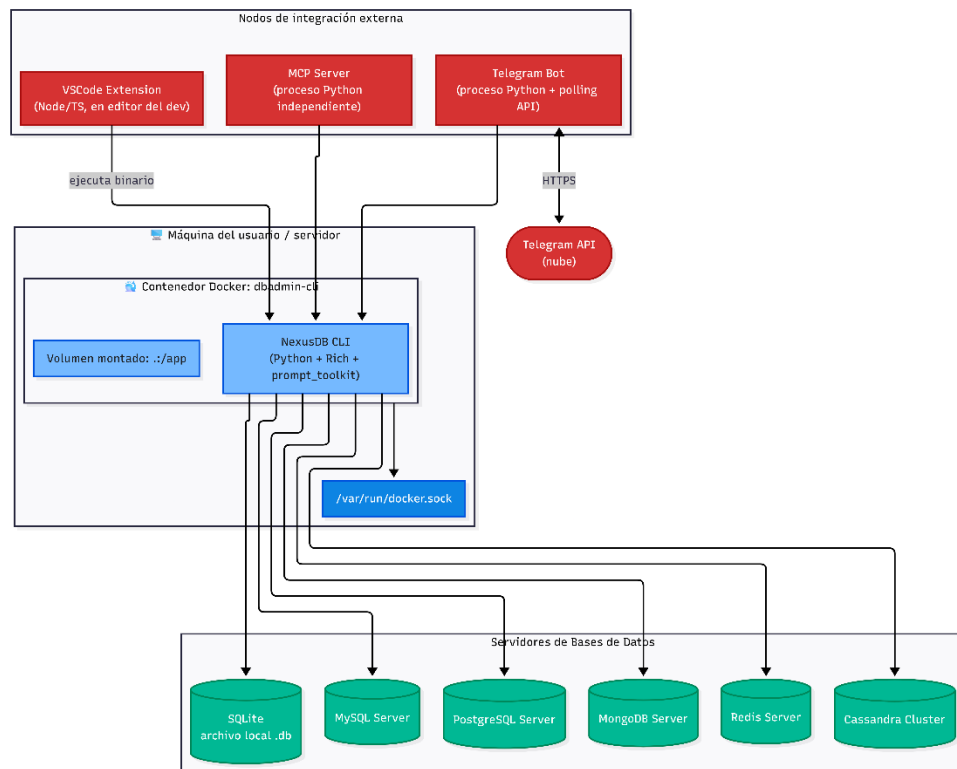
3.4. Vista de procesos

3.4.1. Diagrama de Procesos del sistema (diagrama de actividad)



3.5. Vista de Despliegue (vista física)

3.5.1. Diagrama de despliegue



Además del despliegue local del CLI, el sistema se distribuye en tres entornos adicionales: la extensión de Visual Studio Code (empaquetada con el ejecutable de NexusDB e instalada desde el Marketplace), el servidor MCP (publicado como paquete Python en PyPI e invocado bajo demanda por el cliente de IA) y el bot de Telegram (desplegado de forma permanente como servicio systemd en un VPS Debian, con reinicio automático ante fallos). Los tres canales reutilizan la misma capa de conectores, evitando duplicar la lógica de acceso a datos.

4. ATRIBUTOS DE CALIDAD DEL SOFTWARE.

Escenario de Funcionalidad

El sistema demuestra su funcionalidad al interpretar correctamente los comandos ingresados por el usuario, ejecutar las operaciones CRUD correspondientes sobre la base de datos conectada y mostrar los resultados en formato tabular. Ante comandos inválidos, el sistema muestra mensajes de error descriptivos sin finalizar la ejecución.

Escenario de Usabilidad

Al ser una herramienta de consola, la usabilidad se enfoca en la claridad de los comandos y la legibilidad de los resultados. Se garantiza mediante un comando help que muestra todos los comandos disponibles organizados por categoría, mensajes con indicadores visuales de éxito o error, y resultados presentados en tablas con bordes y estilos.

Escenario de confiabilidad

El sistema previene fallos catastróficos mediante el manejo estructurado de excepciones. Cualquier error de sintaxis SQL, conexión o ejecución es capturado y mostrado al usuario sin que el programa colapse. Las credenciales no se almacenan en archivos ni logs, y las conexiones se cierran explícitamente al desconectar o salir.

Escenario de rendimiento

El bucle REPL y los conectores están diseñados para ejecutar consultas simples en tiempos inferiores a un segundo. El formateo de resultados mediante Rich es eficiente incluso para conjuntos de datos moderados, y la exportación a CSV se realiza de manera inmediata sobre los resultados almacenados en memoria.

Escenario de mantenibilidad

La arquitectura separada en capas (REPL, Conectores, Formateador) y el uso de una clase base abstracta facilitan la extensibilidad. Nuevos motores de base de datos pueden añadirse implementando un nuevo conector que herede de BaseConnector o BaseNoSQLConnector, sin necesidad de modificar la lógica del bucle principal ni del formateador.